HoldON

Live Trip Sharing, Dual-Sensor Panic Tracker & Non-Movement Escalation for Motorcycle Ride-Hailing Passengers

Final Project Proposal · BSCPE 4-6 Group 1

1. Problem Statement

Motorcycle ride-hailing services — locally known as habal-habal and now formalized through platforms such as Angkas and JoyRide — are among the most common forms of daily transport across Metro Manila and provincial cities in the Philippines. They are fast, affordable, and able to navigate congestion that stops larger vehicles entirely. Yet passengers, particularly women riding alone, have virtually no way to let friends or family know where they are in real time, and no discreet mechanism to signal distress if something goes wrong mid-ride.

Incidents of dangerous over-speeding, route deviation, reckless driving, and harassment are well-documented, yet the passenger's only options — shouting, calling emergency numbers, or jumping off — are all impractical or dangerous while the motorcycle is in motion. Meanwhile, family members waiting at home have no visibility into where their loved one is or whether they arrived safely.

HoldOn addresses both problems with a single clip-on IoT device and a shareable web dashboard. Before riding, the passenger sets their origin and destination on the web dashboard, then confirms the trip with a single button press on the device. A shareable link is sent to family or friends — no app installation required — opening a live map with the passenger's moving GPS dot, route, and estimated arrival. If onboard sensors detect a potential emergency — hard grip combined with a sudden jolt, or a deliberate long press of the panic button — all link holders are notified instantly via MQTT → Node.js → Firebase → browser. If the passenger does not move within 60 seconds of that event, HoldOn escalates with a secondary "passenger not moving" alert, repeating every 60 seconds until movement resumes or the trip ends. The trip closes automatically when GPS confirms arrival.

2. Project Objectives

- Enable passengers to set trip origin and destination on the web dashboard before riding, then start the session with a single button press on the device; generate a shareable URL viewable by family/friends with no login or app install required.
- Stream live GPS location from the NEO-7M module to the web dashboard via MQTT → Node.js → Firebase RTDB, updating the passenger's map position continuously while in motion.
- Detect grip intensity via FSR406 analog readings and sudden motion events via MPU6050 accelerometer/gyroscope, triggering a sensor-based panic alert only when both sensors breach their thresholds simultaneously — reducing false positives from road bumps to fewer than 1 in 20 test events.
- Provide a manual panic trigger via long press (≥3 s) of the side button, displaying a 5-to-1 countdown on the OLED with early-release cancel; if held to zero, fire a manual panic alert identical in delivery to the sensor-based alert.
- Deliver browser-based notifications to all trip link holders within 3 seconds of any panic event (sensor or manual) via Firebase Cloud Messaging; flag the incident location on the map.

- Escalate to a “passenger not moving” alert if GPS displacement remains below threshold for more than 60 seconds after a panic event; repeat every 60 seconds with a pulsing red pin on the shared map until movement is detected or the trip is ended.
- Provide LED, OLED, vibration motor, and buzzer feedback on the device for all state transitions: GPS fix, trip active, panic sent, not-moving escalation, movement resumed, and safe arrival.
- End the trip automatically when GPS position enters a 50 m radius of the declared destination, notifying all link holders with a “Safely arrived” confirmation.

3. System Overview & Architecture

The system follows a five-layer IoT architecture: Sensing → Edge → MQTT Broker → Backend → Cloud & Visualization.

Layer	Components	Role
Sensing	FSR406 + MPU6050 + NEO-7M GPS	FSR406 reads analog grip force over a large pad; MPU6050 streams acceleration and gyro over I2C; NEO-7M acquires satellite GPS coordinates offline — no SIM required.
Edge (Device)	NodeMCU ESP8266 + OLED + Green/Red LEDs + Vibration Motor + Buzzer + Side Button	Samples all sensors, runs dual-sensor fusion and button state machine, drives all output components (OLED, LEDs, motor, buzzer), and publishes MQTT payloads to HiveMQ over the passenger’s phone hotspot.
MQTT Broker	HiveMQ Cloud (free tier)	Receives published topics from NodeMCU and routes messages to the Node.js backend subscriber.
Backend	Node.js + Express	Subscribes to all MQTT topics, processes incoming payloads, writes to Firebase RTDB and Firestore, triggers FCM push notifications, and exposes a REST API consumed by the Vue.js dashboard.
Cloud & Visualization	Firebase (RTDB, Firestore, Auth, FCM, Hosting) + Vue.js 3 Dashboard	Firebase stores all trip and event data and delivers push notifications; Vue.js dashboard renders the live map, panic pins, escalation banners, sensor gauges, and trip history — accessible via shareable link, no login needed for recipients.

Full data flow

NodeMCU (sensors + button) → MQTT publish → HiveMQ broker → Node.js MQTT subscriber → Firebase RTDB (live GPS) + Firestore (events) + FCM (push notifications) → Vue.js dashboard (live map + alerts) → link holders’ browsers.

Single-button interaction model

HoldOn uses one momentary push button mounted on the side of the PCB enclosure — positioned away from the FSR406 sensor and natural grip areas so it does not interfere with sensor readings.

Button Action	Device State	Result
Single press	Idle (trip configured on web)	Starts trip; OLED shows TRIP ACTIVE; green LED on; one buzzer beep

Button Action	Device State	Result
Double press (within 400 ms)	Active trip	“Cancel trip?” on OLED; red LED blinks slowly
Single press	Cancel confirmation	Trip cancelled; OLED returns to idle; green LED off; two short beeps
Double press (within 400 ms)	Cancel confirmation	Returns to active trip; no action taken
Long press (≥3 s, hold)	Active trip	OLED shows EMERGENCY 5→4→3→2→1; red LED + vibration motor + rapid beeps during countdown
Release before countdown ends	During countdown	Countdown cancelled; returns to active trip; no alert sent
Hold through full countdown	Countdown reaches 0	Manual panic fires; OLED shows ALERT SENT + coords; red LED solid; three long beeps + vibration burst

OLED screen state flow

Boot → “GPS acquiring…” → “GPS Fixed ✓” → “Trip ready — press to start”

Trip active → GPS coordinates + elapsed time + grip/motion indicators

Panic sent → “ALERT SENT” + coordinates + timestamp

Not moving → “NOT MOVING — alert sent” + time elapsed since panic

Arrived → “Safely arrived!” + destination; trip auto-ends; green LED blinks celebration pattern

Cancel prompt → “Cancel trip? Press once to confirm / Double press to go back”

LED indicator states

LED	State	Meaning
Green	Solid on	Trip active, GPS locked
Green	Slow blink	GPS acquiring / idle
Green	Fast blink pattern	Safely arrived
Red	Solid on	Panic alert sent (manual or sensor)
Red	Slow blink	Cancel confirmation screen
Red	Rapid pulse	Not-moving escalation active
Both	Off	Device idle / no active trip

Trip session flow

1. Passenger opens the web dashboard, logs in, and enters origin and destination.
2. Dashboard generates a shareable link (e.g. holdon.app/trip/abc123); passenger sends it to family/friends.
3. Passenger presses the side button once — trip starts; green LED on.
4. NodeMCU publishes GPS + sensor data to HiveMQ every 5 seconds via MQTT.

5. Node.js backend receives MQTT messages and writes live GPS to Firebase RTDB; dashboard map updates.
6. Panic event fires (sensor or manual): Node.js writes to Firestore, triggers FCM; all link holders notified.
7. Red LED solid; vibration motor burst; OLED shows ALERT SENT.
8. GPS movement resumes within 60 s: map pin updates to “movement resumed”; red LED off.
    a. No movement after 60 s: Node.js sends not_moving FCM alert; red LED rapid pulse; repeats every 60 s.
9. GPS enters 50 m of destination: trip auto-closes; all recipients notified “Safely arrived.”

Dual-sensor fusion logic

The NodeMCU evaluates two conditions every 100 ms. Condition A: FSR406 ADC reading exceeds 700/1023 sustained for 2 seconds — indicating a hard, prolonged grip. Condition B: MPU6050 acceleration magnitude exceeds 2.5 G or angular velocity exceeds 150°/s — indicating a sudden jolt or swerve. A sensor-based panic event fires only when both conditions are true within the same 2-second window. Bumpy roads produce high FSR but low gyro; genuine emergencies produce both.

After any panic event (sensor or manual), the firmware enters a post-panic monitoring state, publishing a “monitoring” MQTT message. The Node.js backend tracks GPS displacement from subsequent location messages. If cumulative displacement exceeds 10 m within 60 seconds, a movement_resumed event is written and monitoring exits. If 60 seconds elapse below the threshold, a not_moving event is written, FCM alert sent, and a new 60-second window begins — repeating until movement is detected or the trip is manually ended.

4. Hardware & Software Components

Hardware

Component	Model / Spec	Purpose / Output Type
Microcontroller	NodeMCU v2 (ESP8266)	Main controller; ADC (A0) for FSR, I2C for MPU6050 + OLED, UART for GPS, GPIO for LEDs + motor + buzzer + button
Grip force sensor	FSR406 (0–10 kg, analog)	SENSOR — measures passenger grip intensity; larger pad improves contact over FSR402
Motion sensor	MPU6050 (I2C, 3.3 V)	SENSOR — 3-axis accelerometer + 3-axis gyroscope; detects jolts and swerving
GPS module	NEO-7M (UART, 9600 baud, ceramic patch antenna)	SENSOR — offline satellite GPS; faster cold start and better urban signal than NEO-6M; antenna included
Display	0.96" OLED SSD1306 (I2C)	OUTPUT (LCD) — shows device state, GPS status, countdown, alerts, and arrival; shares I2C bus with MPU6050
Status LEDs	Green LED + Red LED (3 mm, 3.3 V via resistor)	OUTPUT (LED) — green = GPS/trip status; red = panic/not-moving/cancel states
Vibration motor	Coin vibration motor (3 V, 70 mA)	OUTPUT (MOTOR) — tactile feedback on panic events, not-moving escalation, and countdown; driven via NPN transistor
Audio feedback	Passive buzzer (3.3 V)	OUTPUT — beep patterns for trip start, panic sent, escalation, and arrival
Input button	Momentary push button (side-mounted)	Single-button UI: single press / double press / long press with countdown

Component	Model / Spec	Purpose / Output Type
Voltage divider	10 kΩ resistor	Converts FSR406 resistance to ADC-readable 0–3.3 V range
Motor driver	NPN transistor (2N2222) + flyback diode	Drives vibration motor from 3.3 V GPIO safely
Power	3.7 V LiPo + TP4056 charger module	Portable rechargeable power; charges via micro-USB
Prototype (dev)	Half-size breadboard + jumper wires	Used during development and sensor calibration
Prototype (final)	Custom PCB	Final submission prototype; cleaner wiring, more reliable connections, higher marks

Software & services

Software / Service	Type	Role
Arduino IDE 2.x	C/C++ firmware	NodeMCU firmware: sensor reads, fusion logic, button state machine, OLED + LED + motor + buzzer control, MQTT publish, GPS parse
PubSubClient	Arduino library	MQTT publish to HiveMQ from NodeMCU over Wi-Fi
MPU6050 (jrowberg)	Arduino library	I2C reads of acceleration and gyroscope from MPU6050
TinyGPS++	Arduino library	NMEA sentence parsing from NEO-7M over SoftwareSerial
Adafruit SSD1306 + GFX	Arduino library	OLED display rendering for all state screens and countdown
HiveMQ Cloud	MQTT broker (free tier)	Receives device MQTT publishes and routes to Node.js backend subscriber
Node.js + Express	Backend server	MQTT subscriber (via mqtt npm package); processes payloads; writes to Firebase RTDB + Firestore; triggers FCM; exposes REST API for Vue.js
mqtt (npm)	Node.js library	MQTT client for Node.js backend to subscribe to HiveMQ topics
Firebase Admin SDK	Node.js library	Server-side Firebase access from Node.js: RTDB writes, Firestore writes, FCM sends
Firebase RTDB	Realtime Database	Stores live GPS + sensor readings; streams to Vue.js dashboard sub-second
Firebase Firestore	NoSQL cloud DB	Stores trip sessions and full event log (panic, manual_panic, resumed, not_moving, arrived)
Firebase Auth	Auth service	Passenger account login; trip link holders need no account
Firebase Hosting	HTTPS static hosting	Serves Vue.js dashboard; shareable trip links work in any browser
Firebase Cloud Messaging	Push notification service	Delivers browser push notifications to all trip link holders on panic, not_moving, resumed, and arrival
Vue.js 3 + Vite	Frontend framework	Reactive dashboard: live map, panic pins, escalation banners, sensor gauges, trip history, settings
Leaflet.js	JS mapping library	Live moving GPS dot, route line, static + pulsing panic pins, destination marker

Software / Service	Type	Role
Axios	JS HTTP library	Vue.js frontend makes REST API calls to Node.js backend

5. Backend, MQTT & Cloud Integration

MQTT topics (NodeMCU → HiveMQ)

Topic	Payload	Frequency
holdon/location	{ lat, lng, timestamp }	Every 5 s while trip active
holdon/grip	{ adcValue, level }	1 Hz
holdon/motion	{ accelMag, angularVel }	10 Hz
holdon/event	{ type, lat, lng, gripLevel, accelMag, angularVel, timestamp }	On trigger (panic, manual_panic, monitoring, not_moving, resumed, arrived)

Node.js backend responsibilities

The Node.js + Express server is the central processing hub between the device and the cloud. On startup it connects to HiveMQ as an MQTT subscriber on all holdon/ topics. For each incoming message it: (1) writes live GPS to Firebase RTDB for real-time dashboard streaming; (2) writes structured event documents to Firestore; (3) sends targeted FCM notifications to all trip link holders on panic, not_moving, resumed, and arrival events; and (4) manages the 60-second post-panic movement monitoring window in server memory, publishing not_moving escalations on schedule. The server also exposes a REST API (Express routes) for the Vue.js dashboard to create trip sessions, fetch trip history, and update panic threshold settings.

Firebase — storage & delivery

Firebase RTDB receives live GPS and sensor writes from Node.js and streams them to the Leaflet map in the dashboard. Firestore stores trip session documents and a subcollection of events per trip with types: panic | manual_panic | resumed | not_moving | arrived. Firebase Auth gates the trip-creation flow; the shareable link is public read-only and trip-scoped. FCM pushes browser notifications to all open instances of a trip link, including background tabs, on all event types.

6. Frontend Dashboard (Vue.js)

UI Element	Description
Trip setup screen	Passenger logs in via Firebase Auth, enters origin and destination into a map-autocomplete field. Dashboard calls Node.js REST API to create the trip session and returns a unique shareable URL with a copy button and QR code.
Live trip map (shared view)	Leaflet.js map accessible via the shared link — no login needed for recipients. Shows: animated passenger dot updating every 5 s via Firebase RTDB, route line from origin, destination pin, and ETA.

UI Element	Description
Panic event pin	On any panic event (sensor or manual), a red pin drops on the map at the exact GPS coordinate, labelled with type and timestamp. If movement resumes within 60 s, the label updates to “panic flagged — movement resumed.” The pin is never removed.
Non-movement escalation banner	If GPS displacement stays below threshold for 60 s after a panic, a prominent banner appears: “Passenger not moving — please check immediately.” The map pin switches to a pulsing red indicator. The banner and FCM alert repeat every 60 s until resolved.
Manual panic indicator	Panic events from the long-press button are labelled “Manual panic” on the map pin and in the event log, distinguishing them from sensor-detected events in trip history.
Browser notification	All open trip link instances receive a push notification via FCM within 3 s of panic, not_moving, movement_resumed, or arrival — even when the tab is in the background.
Arrival confirmation	When GPS enters a 50 m radius of the destination, a green banner shows “Safely arrived at [destination].” Trip closes automatically.
Sensor gauges (passenger view)	Logged-in passenger view shows side-by-side radial gauges for FSR406 grip level and MPU6050 acceleration magnitude, updating live via Firebase RTDB.
Trip history	Passenger account lists past trips with replay map, full event timeline, and log of all panic, not_moving, resumed, and arrival events with timestamps and coordinates.
Settings panel	Adjustable thresholds (FSR ADC level, G-force, hold duration, not-moving repeat interval) stored in Firestore — updated via Node.js REST API without reflashing firmware.

7. Development Timeline (6 Weeks)

Week	Focus	Deliverables
Week 1	Hardware & sensor wiring (breadboard)	Wire FSR406 (voltage divider → A0), MPU6050 (I2C), NEO-7M (SoftwareSerial), SSD1306 OLED (I2C), green + red LEDs (via resistors), coin vibration motor (via NPN transistor), passive buzzer, side button (pull-up) on breadboard. Verify all sensors and outputs independently via Serial Monitor.
Week 2	Firmware — sensors, fusion logic, outputs & button	Implement FSR406 classifier and MPU6050 magnitude computation. Code dual-sensor AND panic logic. Implement single-button state machine (single / double / long-press countdown). Drive OLED state screens, LED patterns, vibration motor bursts, and buzzer beep sequences. Publish all MQTT topics to HiveMQ over Wi-Fi hotspot.
Week 3	Node.js backend — MQTT subscriber + Firebase writer	Set up Node.js + Express server with mqtt package subscribing to all holdon/ topics. Implement Firebase Admin SDK writes to RTDB (live GPS) and Firestore (events). Implement 60-second post-panic monitoring logic and not_moving escalation in Node.js. Set up FCM notification sends. Test full pipeline: physical grip + jolt → HiveMQ → Node.js → Firebase.
Week 4	Vue.js — trip setup, REST API & shared live map	Build Node.js REST API routes (create trip, fetch history, update settings). Build Vue.js trip setup screen (origin/destination + shareable link + QR code). Build shared map view with live moving dot via Firebase RTDB SDK. No-login guest access.
Week 5	Dashboard — alerts, escalation, arrival & history	Add panic pin (sensor + manual variants), escalation banner, and movement-resumed logic. Add arrival detection (50 m radius, auto trip close). Build trip history + sensor gauges + settings panel. End-to-end test: device → MQTT → Node.js → Firebase → Vue.js → browser notification.

Week	Focus	Deliverables
Week 6	PCB layout, testing & deployment	Transfer breadboard circuit to PCB layout; solder and verify all connections. 50-event stress test: sensor panic accuracy, manual panic, not-moving escalation, false positive rate, alert latency, LED/motor/buzzer outputs. Firebase Hosting + Node.js deployment. Demo rehearsal and documentation.

8. Success Criteria

- All 4+ output types function correctly: OLED updates state on every transition, green/red LEDs match the correct state patterns, vibration motor fires on panic and escalation events, and buzzer plays the correct beep sequence for each event — verified across all states.
- Shareable trip link opens a live map within 3 seconds on a standard mobile browser with no login or app install required.
- Single-button interaction works correctly: single press starts, double press opens cancel prompt, long press triggers countdown, early release cancels, full hold fires manual panic — verified across 20 interaction tests.
- Live GPS dot updates on the shared map within 5 seconds of the passenger moving, verified over 10 outdoor test rides via the full MQTT → Node.js → Firebase → dashboard pipeline.
- Dual-sensor panic accuracy ≥ 90%: at least 45 of 50 combined grip + jolt events trigger an alert; fewer than 1 in 20 bump-only or turn-only events produce a false positive.
- Manual panic fires on full long-press countdown; early release produces no alert in 10 of 10 tests.
- End-to-end panic alert latency ≤ 3 seconds: from sensor breach or countdown completion to browser notification on a shared trip link tab, measured over 10 trials.
- Non-movement escalation fires within 3 seconds of the 60-second window closing; repeats every 60 seconds; verified across 10 consecutive stationary test events.
- Movement-resumed pin update occurs within 60 seconds of GPS movement resuming, verified in 10 consecutive test events.
- Arrival detection fires and auto-closes the trip when GPS enters 50 m of the destination, confirmed in at least 8 of 10 outdoor approach tests.
- PCB prototype passes all functional tests above with no loose connections or short circuits; all components soldered and housed in the clip-on enclosure.

Members:
Almario, Candido James C.
Bayas, Alleah Marie G.
Benciang, Gienel Aubrey V.
Convicto, Justine Mae L.
Francisco, Eloissa S.
Macatangay, Keanno Brennel A